

A Domain-Specific Language for Prior-Art Search Workflow in Patent Examination

Shanaya Malik
shanaya_malik@berkeley.edu
University of California, Berkeley
Berkeley, California, USA

Kelvin Zhao
kelvinyz@berkeley.edu
University of California, Berkeley
Berkeley, California, USA

Abstract

Prior-art search requires patent examiners to repeatedly switch among corpus scoping, targeted search, close reading, and claim-to-evidence mapping. However, current tools primarily return ranked documents, leaving section selection, citation recovery, and claim-chart drafting to manual work. To better understand what a better abstraction should support, we conducted a contextual inquiry study with 10 current and former United States Patent and Trademark Office (USPTO) patent examiners and identified recurring needs for section-aware retrieval, pinpoint citation, iterative query refinement, and tighter integration between search and synthesis. We present a domain-specific language for structured prior-art search in which each patent is represented as a Document → Section → Passage hierarchy, queries compose structural, textual, and metadata predicates over that hierarchy, and matched results carry provenance forward into a connected evidence-capture environment for claim-chart drafting. Our prototype combines a FastAPI backend, a query execution model over parsed patent passages, and a React frontend for corpus scoping, live refinement, and lightweight claim-chart drafting. The paper contributes the design rationale, implementation, and technical challenges of treating post-retrieval patent analysis as an inspectable, programmable workflow rather than ad hoc document inspection.

CCS Concepts

• **Human-centered computing** → Empirical studies in HCI; HCI design and evaluation methods; • **Software and its engineering** → Domain-specific languages.

Keywords

patent examination, prior art search, domain-specific language, document analysis, contextual inquiry, sensemaking, information work, HCI

ACM Reference Format:

Shanaya Malik and Kelvin Zhao. 2026. A Domain-Specific Language for Prior-Art Search Workflow in Patent Examination. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email*

(Conference acronym 'XX). ACM, New York, NY, USA, 17 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Motivation

Patent examination is a specialized, document-intensive activity in which an examiner must locate prior art — existing patents, publications, or other disclosures — that *anticipate* or *obviate* each claim in a pending patent application [1]. The quality of the examination directly affects the validity of the intellectual property record and the downstream litigation and re-examination costs. However, the core task of finding prior art remains poorly supported in that an examiner reads the claims, identifies the key technical concepts, formulates search queries, retrieves candidate documents, reads those documents to find relevant passages, and then records exactly *where* in each document the evidence appears so it can be cited in an office action. Each of these steps is handled by a different tool or no tool at all, and the transitions between them—from search to close reading, from close reading to citation, from citation to claim mapping—are left entirely to the examiner. For complex technical art units—software, biomedical devices, quantum computing—this cycle can span dozens of documents and hundreds of individual claim elements per application.

The current search infrastructure—including the USPTO's internal Examiner Automated Search Tool (EAST) and Web-based Examiner Search Tool (WEST), as well as commercial tools such as PatSnap and Derwent Innovation—treats this workflow at the document level, returning ranked lists of patents rather than the passage-level evidence examiners actually need [1]. A query returns a ranked list of patent documents, and the examiner then opens each document and manually locates the relevant section. The tools surface keyword hit *counts*, but do not distinguish whether a hit appears in the abstract, background, detailed specification, or the claims—sections that carry very different evidentiary weight. Passage-level retrieval, section-aware filtering, and metadata-constrained queries are not available as first-class capabilities in any tool accessible to working examiners.

To understand where current tooling breaks down in practice, we conducted a contextual inquiry study with ten practicing patent examiners and identified five recurring needs that cut across the ordinary path from reading a claim to writing an office action: synonym generation, filing-date filtering, claim-mapping handoff, section-aware passage inspection, and iterative query refinement. For example, P05 explicitly wanted hits in the background and detailed description sections rather than the abstract or claims, because the specification is where the technical teaching that maps to claim elements is typically found. P03 showed that determining which references were even legally admissible could require

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

tedious manual date inspection across candidate documents. P01 emphasized the effort required to recover exact paragraph-level locations for later citation, and multiple participants described the move from search into claim mapping as a separate, mostly manual activity carried out in Word, paper notes, or personal templates. These observations highlight that critical legal-technical judgments are being made through brittle manual transitions, motivating us to address this problem.

The five needs identified above share a common structural limitation: current tools treat patent documents as opaque text blobs; the rich internal structure of a patent — its sections, numbered paragraphs, claim hierarchy, and embedded metadata — is invisible to the query layer. An examiner who wants to find all passages in the detailed specification of a particular assignee's patents that discuss a specific technical mechanism must open each document, scroll to the relevant section, and scan manually. This is not a failure of individual tools but a consequence of the retrieval model: when documents are the unit of retrieval, there is no path to passage-level precision.

The reason this gap persists is that patent search has historically been framed as a retrieval problem rather than a workflow problem, leading existing systems to optimize for coverage and ranking while leaving the harder coordination work to examiners. Existing systems compete on coverage, ranking, and document access, which makes sense if the main question is which references to open next. Our study suggests that this framing is too narrow for examiner work. Examiners do not stop at document retrieval: they must interpret where relevant teaching appears, verify admissibility, preserve citation anchors, and organize evidence against claim elements. Those steps span information retrieval, document structure, and downstream synthesis, so they often fall between existing tool categories.

This gap has also been difficult to close because the workflow itself is hard to study. Much examiner practice occurs within specialized institutional settings and proprietary tool ecosystems, which means the downstream coordination work—the synonym management, date verification, and claim mapping that follow retrieval—is easier to miss than the search box itself [1]. Prior work on patent retrieval has consequently focused on what is most visible and measurable: coverage, ranking, and recall [2]. The interpretive and synthesis-oriented stages that our study surfaces have received much less attention, and no prior tool has been designed around the full arc from corpus scoping through evidence capture.

We propose a different approach, which is that rather than building another document-level search interface, we present a domain-specific language for structured prior-art search in which each patent is represented as a Document → Section → Passage hierarchy and queries compose structural filters—section:SPECIFICATION, paragraph:0042—with content filters and metadata constraints using Boolean operators. The language executes at the passage level and returns matched passages with their full provenance, including section type, paragraph anchor, and neighboring context. This language is embedded in an integrated analysis environment that supports corpus scoping before search and carries matched passages forward into a claim-chart workspace,

treating retrieval and synthesis as parts of the same workflow rather than forcing a handoff to external notes or templates.

If successful, this approach shifts the unit of work from opening documents and manually reconstructing context to inspecting, refining, and reusing structured evidence. Examiners can keep the active corpus explicit, express search logic in a reusable form, see why a passage matched, and carry that passage forward with its citation anchor and query provenance intact. The intended outcome is not a fully automated examination; it is a workflow in which expert judgment is still central, but the surrounding search and evidence-management machinery is more inspectable, reproducible, and better aligned with how the work is already done.

The key PL contribution is the abstraction this gives users. Instead of issuing opaque keyword searches over whole documents, examiners program against an explicit representation of a selected corpus of structured patent objects, an executable query expression over sections and passages, and an evidence record that carries both the retrieved passage and the program state that produced it. This is the sense in which our system is a programming intervention rather than only a search interface: it exposes a new language and execution model for a domain task that was previously performed through ad hoc combinations of search, scrolling, and manual note-taking.

The design of our DSL is motivated by two key insights from our need-finding. First, patent examiners are not casual users: they are technical specialists who write Boolean search strings as a core job skill, and several participants described current query interfaces as too limited rather than too complex (P04, P07, P10). A textual DSL, therefore, appears appropriate for this population in a way it might not be for a general consumer audience. Second, the iterative, refinement-based search strategy that examiners actually use (Need 5) maps well to what a composable query language supports: an examiner can start with a broad section:CLAIMS filter, observe the results, narrow with AND contains: "neural interface", and further restrict with AND NOT paragraph:0001 to skip boilerplate preambles. Each refinement is a surgical edit to the query string, not a round-trip through a separate GUI tool.

The contributions of this paper are as follows:

- We present a domain-specific querying language that combines corpus scoping, structured passage-level querying, and evidence capture for claim-chart drafting, reframing prior-art search as a connected analysis workflow rather than a document-level retrieval task followed by manual synthesis.
- We describe the passage-level document model and composable query DSL that realize this environment, exposing section type, numbered paragraph anchors, claim structure, and patent metadata as first-class query targets grounded in the needs identified through contextual inquiry.
- We present a prototype implementation—a FastAPI backend with a React frontend—that parses USPTO-format patent documents (both plain-text and text-extractable PDF), executes passage-level queries over a scoped corpus, and carries matched evidence into a lightweight claim-chart workspace.

- We characterize the open technical challenges in this design space, including document parsing fidelity, query expressiveness limits, and the performance tradeoffs of passage-level indexing and interactive corpus scoping at larger scales.

2 Related Work

Our work draws on four overlapping areas of prior research: sensemaking and exploratory information seeking, patent retrieval and document-analysis tools, structured query languages, and contextual inquiry for specialized workflows. The consistent gap across prior work is that it has more to say about how to retrieve, rank, or study information work than about how to carry fine-grained evidence forward into the structured synthesis that expert patent analysis requires.

2.1 Sensemaking and Information Foraging in Professional Technical Work

The cognitive demands of patent examination have strong parallels in prior work on sensemaking and information foraging. Pirolli and Card describe complex analysis as movement between a foraging loop, in which analysts gather potentially relevant information, and a sensemaking loop, in which they structure and interpret what they have found [3]. Russell et al. similarly argue that much of the cost of complex information work lies not only in finding information, but in organizing it into representations that support judgment [4]. Patent examination follows exactly this pattern: examiners search for potentially relevant prior art, but the harder step is often determining which passages matter, how they relate to specific claim elements, and how to preserve those judgments for office-action writing.

This makes patent examination closer to exploratory search than to simple lookup, where Marchionini characterizes exploratory search as search undertaken to learn, investigate, and iteratively refine understanding rather than simply retrieve a known fact [5]. Bates's berrypicking model and Kuhlthau's information-search-process model similarly emphasize evolving queries, reformulation, and changing understanding over the course of inquiry [6, 7]. White and Roth argue that exploratory search systems should support users as they formulate, reformulate, organize, and compare information over time, while Hearst emphasizes faceting, query reformulation, and result inspection as central parts of interactive search rather than secondary interface details [8, 9]. These perspectives help frame our participants' behavior: examiners repeatedly changed terminology, narrowed the active corpus, and compared passages in context before deciding what evidence was worth reusing.

Related observations appear in software engineering and other professional technical domains. Ko et al. found that developers navigating large codebases must repeatedly search for scattered information and reconstruct reasoning chains across files [10]. Storey et al. showed that tool designers often assume a single cognitive strategy even though expert users alternate among top-down, bottom-up, and opportunistic approaches [11]. Nersessian's account of model-based reasoning likewise emphasizes that expert work depends on constructing and manipulating intermediate representations, not just retrieving facts [12]. These studies help explain why document

retrieval alone is not enough for patent examination: the work also requires preserving context, structuring evidence, and supporting iterative reinterpretation.

At the same time, prior sensemaking research does not directly address the particular combination of legal interpretation, technical comprehension, and evidentiary traceability that defines patent examination. Our work builds on these studies by treating patent analysis as a form of structured document work in which retrieval, interpretation, and evidence reuse must be supported within the same environment. In that sense, this strand explains why the work is difficult, but not yet what a tool should preserve as the unit of reusable evidence.

2.2 Tools for Patent Search and Document Analysis

Work on patent retrieval has historically focused on search coverage, ranking, classification, and recall-oriented retrieval. Lupu et al. survey this space and show that while patent search systems have improved substantially as retrieval engines, relevance assessment and downstream analytical support remain open problems, and that continues to describe the landscape of many public and commercial systems [1]. Tools such as Google Patents, PATENTSCOPE, Derwent Innovation, PatSnap, and the USPTO's own search environments provide large-scale patent access, classification search, citation links, and, in some cases, semantic or AI-assisted retrieval. However, they still center the workflow on retrieved documents rather than on passage-level evidence objects tied to claim elements.

The research literature around patent retrieval shows that work in the CLEF-IP community helped establish patent retrieval as a benchmark-driven information-retrieval problem, especially around prior-art search, multilingual retrieval, and evaluation methodology [13]. Other work focused on improving retrieval effectiveness through better query formulation over structured patent documents [14] and through query expansion techniques such as citation analysis [2]. These contributions were important because they treated patents as a distinct IR domain with unusual document structure and difficult relevance judgments. At the same time, they mostly optimized which references should be retrieved or ranked, not how an examiner should inspect, preserve, and reuse specific passages once a promising reference has been found.

The broader search-interface literature makes that limitation clearer. Interactive search systems often improve access through facets, query suggestions, aggregation, and result previews [8, 9], but those gains do not by themselves address the downstream evidentiary work patent examiners must perform. After retrieval, examiners still need to determine where relevant teaching appears, whether it occurs in the right section of the patent, whether the source is legally admissible, and how the resulting evidence should be mapped to claim limitations. This means that while a tool can be a strong retrieval interface, it can still leave the hardest interpretive and synthesis steps outside the system. Our findings therefore align with Lupu et al.'s observation that assessment remains a weak point, and we extend it in a more workflow-oriented direction: what is missing is not only better ranking, but better support for the interpretive and synthesis-oriented work that follows retrieval.

Outside the patent domain, a related design space appears in systems that help users work across large structured corpora rather than isolated documents. Program comprehension tools, for example, often help developers move among related artifacts while maintaining context, but they rarely need to preserve citeable evidence for downstream legal-style reasoning [10, 11]. Our DSL and its surrounding environment differ from both patent search tools and cross-document analysis tools in that they connect passage retrieval, structural provenance, and evidence capture into one continuous workflow. The key distinction is therefore that prior systems mainly stop at retrieval quality, whereas our design treats retrieval and evidence assembly as part of the same tool problem.

2.3 Domain-Specific Languages for Structured Document Access

Our tool also draws on a long tradition of structured query languages and retrieval systems that expose domain structure rather than hiding it behind a single keyword box. Chamberlin and Boyce's SEQUEL showed that a query language can expose a compositional, human-readable interface over structured data while remaining expressive enough to encode complex information requests [15]. Later work extended that idea from relational tables to structured document collections: XQuery, for example, was designed to query XML documents and collections through an explicit data model rather than raw full-text search [16]. In a different line of work, corpus query systems such as the IMS Open Corpus Workbench exposed dedicated query languages over large annotated text corpora, making token-level and structural constraints first-class parts of corpus search [17]. The core idea relevant to our work is the broader design principle that a domain-specific query language can serve as both an interaction medium and a reusable artifact.

This strand of work also connects to the exploratory-search literature above. If users refine their understanding by making local, inspectable changes, then a query language is valuable because it gives the user a persistent representation of search intent that can be edited and reused [5, 8]. That framing helps explain why our project uses a textual DSL rather than only a point-and-click filter panel: the query itself becomes part of the reasoning artifact, not just a transient UI state.

There is also a connection to structured document retrieval. Piwowarski and Dupret argue that classical information-retrieval models are poorly matched to settings where retrievable units are document parts and where users may need to navigate among related structural elements [18]. That concern is directly relevant to patents, where the useful unit is often not the whole document but a passage in a particular section with a citeable anchor and nearby context. Our work can therefore be understood as bringing a structured-query perspective together with a structured-retrieval perspective: the DSL denotes sets of passages over a patent hierarchy, not lists of opaque documents.

In comparison to prior structured query languages, which are typically designed around a data model or retrieval formalism, our DSL was shaped by direct observation of examiner workflows. The `section:`, `paragraph:`, `metadata`, and `Boolean` constructs are

design responses to observed examiner practices such as section-scoped reading, synonym expansion, filing-date checks, and iterative refinement. In this sense, the language is both a PL artifact and an HCI artifact: its semantics are tied to the document model, and its form is tied to the work that users already do. Relative to prior structured-query work, the novelty here is therefore not just that the language targets a new corpus, but that it is bound to a provenance-preserving workflow derived from field observations. Among the systems we reviewed, we did not find prior query languages that combine section-aware passage retrieval over patent documents with provenance-preserving evidence handoff into claim mapping.

2.4 Contextual Inquiry for Studying Specialized Workflows

The need-finding component of this work is grounded in the contextual inquiry framework of Beyer and Holtzblatt, using the Master-Apprentice model to surface tacit knowledge and workflow breakdowns through direct observation [19]. This approach is especially valuable in specialized domains because practitioners often rely on tacit routines and workarounds that are hard to surface through retrospective interviews alone. Ko et al. used observation rather than purely self-reported accounts to study how developers gather and relate information during maintenance work [10]. Lubin and Chasins likewise used direct observation to reveal patterns in functional programmers' practice that would be easy to miss in a survey-only design [20]. Zhang et al. demonstrate a related need-finding orientation in their study of API design, showing how unmet information needs can guide the design of technical tools and interfaces [21].

The observational findings from our contextual inquiry study serve as a foundation for the design of our working DSL and integrated environment. The methodological contribution of the paper is therefore not only that CI can reveal unmet needs in patent examination, but that those needs can be carried forward into the design of a domain-specific representation, language, and environment. Reflexive thematic analysis provides the mechanism by which those observations are consolidated into design-relevant themes rather than treated as isolated anecdotes [22]. This strand therefore justifies the paper's design method, while the earlier strands help show why the resulting tool must integrate retrieval, inspection, and evidence reuse rather than treating them as separate stages.

3 Need Finding

3.1 User Study Design and Procedure

The study was designed to address the research question: *What kinds of problems do technical patent examiners face during the iterative process of discovering, filtering, and synthesizing prior art to evaluate the novelty of complex technical claims?* This question was scoped to examiners in technical art units (such as software, biomedical engineering, or quantum computing), where the demands of translating between legal claim language and technical implementation are particularly demanding. We chose not to pre-specify hypotheses, since the study was exploratory, need-finding.

The participants were recruited through purposive and snowball sampling, beginning with two professional contacts who shared

a recruitment message with colleagues and provided us with additional contacts. To supplement the remaining number of participants, we used cold messaging on LinkedIn to target individuals whose profiles indicated professional experience in technical patent examination. More specifically, the inclusion criteria required participants to be at least 18 years old, fluent in English, and currently or formerly employed in patent examination for the USPTO, with experience evaluating complex technical claims. We excluded individuals who exclusively examine non-technical patents and anyone unable to demonstrate their workflow using only publicly available documents.

Each session lasted approximately 45 minutes and was conducted remotely via Zoom. The sessions began with a brief unrecorded introduction of the participant and researcher, followed by an overview of the project scope and agenda for the session. The core of the session (~30–45 minutes, recorded) was a contextual inquiry following the Master-Apprentice model, in which the participant shared their screen and demonstrated their prior-art search and synthesis process for a given public patent while thinking aloud, with clarifying questions asked at natural pause points [19]. This was followed by a synthesis segment focused on how the participant maps and documents the relationship between prior art and specific claim elements, and concluded with a closing discussion addressing the most time-consuming and mentally demanding aspects of the workflow.

Each session produced three data artifacts stored on a password-protected Berkeley bDrive account: (1) a Zoom screen-and-audio recording capturing screen activity, search queries, and think-aloud narration; (2) typed session notes taken in real time; and (3) a post-session debrief document recording emerging patterns. The recordings were transcribed within Zoom itself and reviewed for accuracy. The participants were assigned coded identifiers (P01, P02, etc.), and we consolidated each participant's transcript, session notes, and debrief into a single document as part of our input to our reflexive thematic analysis [22].

3.2 Analysis Process

We analyzed the consolidated session documents using reflexive thematic analysis following Braun and Clarke [22]. Each researcher independently conducted an initial round of open coding on their assigned participant's contextual inquiries, applying purely descriptive labels to segments that capture actions, reasoning, breakdowns, and workarounds within each patent examiner's workflow. Since our data include both think-aloud narration and question answers, as well as screen-recorded behavior, our coding explicitly distinguishes between what participants said and what actions they took, allowing us to capture both direct information from explanations and implicit information from instances where difficulties were not verbally articulated. As a result, many codes described domain-specific interactions, such as refining search queries, tracking claim coverage across references, and using specific internal tools to manage information across documents.

After independent coding, we compared our code sets across assigned participants and iteratively refined them through discussion, focusing first on resolving differences in interpretation and

initial grouping, before collaboratively identifying higher-level patterns that showed recurring needs across multiple participants. We treated this stage as an iterative process, merging overlapping codes and preserving any meaningful differences reflecting underlying issues in the participants' workflows. We grouped related codes into candidate themes, tightened their definitions, and verified that each theme was supported by evidence from multiple participants. The five themes we present below represent the needs that emerged most consistently across our participant pool.

3.3 Findings

Need 1: Examiners lack systematic support for generating and managing the synonyms needed to translate legal claim language into effective search queries. A central challenge across sessions was the gap between claim terminology and the terminology used to retrieve relevant prior art; as P04 explained, the applicant “comes up with their own term, so we try to come up with terms that are going to be describing the same thing.” P06 described this as a problem of implied language: a term like *modder* in a medical device patent could return results about computer modification, and a term like *wearable* might not appear in a relevant document that uses a synonym, meaning those documents would be filtered out. Examiners relied on a patchwork of ad hoc resources, including a shared synonym crosswalk tool (P04), Google searches (P04), terminology from prior specifications, and personal experience. P06 described a case where an applicant's term for an ophthalmic structure appeared only in their own application, requiring consultation with a senior examiner to identify standard terminology. P07 noted that while examiners are taught about synonyms and Boolean connectors, applying this in unfamiliar art areas requires substantial independent research, alongside occasionally having to consult a senior examiner. Similarly, P10 described getting “stuck with the synonyms” when attempting to expand search queries in unfamiliar or highly technical applications, reinforcing that synonym generation remains an unsupported and cognitively demanding step. Junior examiners were particularly disadvantaged, lacking the tacit vocabulary that senior examiners described as built through years of repetition (P01, P05, P06).

Need 2: Current search tools do not adequately support filtering by filing date, leading to wasted effort and rework. Every participant referenced the effective filing date as a hard constraint on admissible prior art, yet existing tools provided limited support for enforcing it during search. P05 described completing an entire office action based on a reference that was found to post-date the application's effective filing date, necessitating a complete redo. P06 identified a related problem in which search systems frequently confuse publication dates with filing dates, and also confuse non-provisional filing dates with earlier provisional dates that establish actual priority. P06 called this “definitely a very good fix that can be done.” P01 noted that the rules for effective filing dates differ between U.S. and foreign applications, making even manual checks non-trivial. P08 described the filing-date constraint as deeply embedded in the workflow, requiring cross-referencing across domestic, provisional, and foreign filings, and P03 had initially demonstrated this in action, showing that of seven initial

results, only one had a usable date (but determining this required a manual, time-consuming inspection of each).

Need 3: The transition from search to claim mapping relies on entirely manual, ad hoc methods with little integrated tool support. After identifying promising prior art, examiners must document how specific passages address specific claim elements, which is a process called claim mapping. Every participant performed this using personal, improvised methods outside the search tool. P01 maintained a Word table; P03 copied claims as bullet points and mapped inline; P04 used a personal claim chart; and P05 copied claims into a Word document with parentheses after each element and drew dependency trees by hand. P06 described using pen and paper for claim trees and explained that the in-house software's mapping feature was so inaccurate that "pretty much no one at the USPTO" trusts it—errors in claim dependencies can distort the entire analysis. P07 described the process as lacking elegant tooling, stating that "it's a very manual process, [examiners] just start plopping it in there." P08 described printing claims on paper and annotating them physically to manage cognitive load across simultaneous cases, and P05 explicitly identified this as a design opportunity, stating that integrating claim mapping into the search tool could save a significant amount of time and energy for examiners.

Need 4: Examiners struggle to locate and interpret relevant information within retrieved documents. Examiners reported that tools display keyword hit counts without distinguishing whether hits appear in the specification (the detailed description most useful for mapping), the abstract, or the claims. P05 wanted hits specifically in the background and description sections, not in the abstract or claims, and shared that the ranking system orders results by total hit count regardless of location, forcing examiners to open documents and manually scan for relevant sections. P02 used a similar process of scrolling and combing through CTRL+F results of keywords to find relevant paragraphs. P01 expressed further frustration with granted patents, which require citing by column and line number rather than paragraph number, adding manual overhead when recording where evidence appeared. P09 described that some internal tools support multi-highlighting to track keyword locations across claims, but public tools lack this capability entirely, forcing examiners to flip between separate documents to locate where relevant content appears manually. In practice, P10 also described relying on scanning titles, images, and highlighted keywords to triage relevance, further illustrating that examiners must manually piece together signals across document representations to determine whether a result is worth deeper inspection. Several participants also described needing to read beyond the keyword hit itself to determine whether it was actually meaningful in context, often revisiting multiple sections of the document to interpret how the term was being used. For example, P03 explained that when a result appears potentially relevant, they cannot rely on keyword matches alone and instead "have to go through the whole patent" to verify whether it actually aligns with the claim, since surface-level indicators like keyword proximity are often insufficient. Together, these patterns suggest that relevance assessment is slowed not only by the difficulty of locating keyword

matches but also by the additional cognitive effort required to interpret their significance in complex technical documents.

Need 5: Existing AI-assisted search tools do not align with the iterative, refinement-based search strategy that examiners actually use. Participants with experience using AI or semantic search tools—including the USPTO's similarity search, IP.com, and other platforms—consistently described them as inadequate. This was also visible in observed behavior: in one session, a participant invoked similarity search, briefly inspected the large returned set, and then abandoned that path in favor of manual Boolean refinement once it was clear that the suggested references were not preserving the technical distinctions needed for the task. P02 emphasized that effective search "really involves appraising the art as you search for it, and refining your search based on that," which current tools do not support. P06 described the similarity search as "hit or miss," identifying the core problem as the tool's inability to distinguish domain context—a medical-device search for "user interface" returns results from banking and unrelated software. P07, while noting that IP.com "seems to interpret language a little bit more intuitively," still gravitated toward non-patent sources, such as technical manuals, as a more efficient strategy. P09 echoed the mixed assessment, noting that the similarity search "has mixed results—sometimes I just click that button... and it finds me 300 references, of which 2 or 3 are actually useful. Other times, it's just completely off the mark." P09 also described a workaround of feeding publicly available PG pubs into external AI tools to help devise search strategies, while noting that proprietary restrictions prevent the use of AI with actual application information. It appeared that, across most participants, AI tools were supplements rather than primary methods, reinforcing that future design should augment iterative workflows rather than replace them.

3.4 Implications for Future Research and Tool Design

Several of our findings align with patterns reported in prior work on professional information work, including the challenges of maintaining context across documents [10], organizing scattered evidence [4], and iterative refinement in expert search [3]. However, our findings also diverge from prior work in important ways. First, while Ko et al. [10] found that developers lose track of reasoning chains across files, patent examiners face an additional structural constraint: their reasoning must be organized element-by-element against legally defined claim language, not simply against a mental model of the codebase. The failure mode is therefore not just lost context but legally indefensible analysis. Second, Lupu et al. [1] identified relevance assessment as an open problem in patent retrieval, but framed it primarily as a ranking and interface problem; our observations suggest that a deeper issue is the absence of integrated support for the interpretive work that follows retrieval—claim mapping, synonym management, and filing-date verification—which existing tools do not address at all. Third, Storey et al. [11] observed that developers alternate among cognitive strategies and argued that tools should accommodate this flexibility; we find a similar pattern among examiners, but with the added constraint that the output of their sensemaking must conform to a rigid legal structure, limiting how flexibly they can reorganize their analysis. Finally,

while Pirolli and Card’s sensemaking model [3] describes foraging and sensemaking as iteratively coupled, our participants described AI-assisted search tools that treat these as separate, one-shot operations (Need 5), suggesting that current tool design may work against the iterative workflow that prior theory predicts experts will use.

These differences suggest that future research should more explicitly account for domains where information work is not only interpretive but also constrained. Our findings also suggest that future tool design could prioritize integration over automation. Rather than replacing examiner judgment with AI-driven search, promising opportunities may lie in reducing manual overhead for tasks examiners already perform, such as synonym management, date filtering, and claim-to-evidence mapping.

4 Design of the Tool

4.1 Overview

The core of the tool is a textual DSL that queries parsed patent documents at the passage level rather than the document level. The language operates on a structured representation in which each source is modeled as a Document → Section → Passage hierarchy with preserved patent metadata, section type, numbered paragraphs, claim anchors, and neighboring context. Instead of treating patents as opaque full-text blobs, the DSL lets an examiner scope a working corpus through explicit document selection, metadata facets, or metadata predicates and then express passage-level constraints over that structured document representation. The system executes those queries at the passage level and returns matched passages with provenance: document title and identifier, section type and heading, passage index, optional paragraph or claim anchor, contextual neighbors, and a human-readable reason string explaining why the result matched. The broader environment combines corpus scoping, structured querying, and evidence capture for claim-chart drafting. Rather than replacing the examiner’s initial document-level prior-art search, the tool is designed to operate immediately afterward, once an examiner has already identified a candidate set of potentially relevant patent references.

The environment has two connected surfaces which include (1) the primary search page, and (2) the claim-chart page. The main search page combines a document manager for corpus scoping, a DSL editor for passage-level queries, and a scrollable list of result cards. Each result card supports lightweight downstream actions: the examiner can copy a citation trace directly from the card or send the passage into a separate claim-chart workspace. The second surface is a grouped claim-chart page in which evidence rows are organized under claim elements, edited, and exported as a TSV or DOCX file. The second surface remains prototype-grade, but it is included intentionally because our study showed that the handoff from search to claim mapping is itself a significant gap in current examiner tooling (Need 3). Figure 1 shows the search workspace, where corpus scoping, query authoring, and passage-level inspection remain visible together.

The central user-facing abstraction is not just a better search box, but a program state composed of three linked parts: (1) a scoped corpus, (2) a query program, and (3) a set of evidence artifacts.

The scoped corpus is the selected patent set plus its metadata constraints. The query program is the DSL expression that denotes a set of matching passages over the Document → Section → Passage hierarchy. The evidence artifact is not just copied text; it preserves the selected passage, its citation anchor, the source document set, and the query text that produced it. This gives users something closer to a reusable, inspectable representation of search intent and search outcome that can move forward into claim-chart drafting and backward into result reproduction.

The search interface supports both explicit submission and live refinement. On a fresh page load, the system does not immediately run a broad query across all selected documents; this avoids hydrating unnecessary documents before the examiner has begun interacting. Once the examiner edits the query, changes the document selection, or presses the run button, the interface enables a 600ms debounced live-refresh loop so subsequent refinements update the result set in near real time. In parallel, the frontend opportunistically preloads the currently selected documents so the first query on that selected set need not hydrate each document one by one.

These design choices are specific to the constraints we identified in contextual inquiry with patent examiners. We therefore organize the design not as a list of interface features, but as a mapping from observed workflow breakdowns to design requirements and then to implemented tool responses.

4.2 Walkthrough: Using the Tool for a Real Claim-Mapping Task

In order to make the design concrete, we walk through a task modeled on one that was observed in session P03, where the patent examiner was evaluating a software patent application whose claim required a mechanism for *dynamic routing table updates*. The task was not simply to find documents that mentioned routing tables. P03 needed to identify which candidate references were legally admissible, determine whether the relevant teaching appeared in the specification rather than only in the claims or abstract, inspect the surrounding context of each hit, and then preserve the best evidence for claim mapping. In the observed workflow, these steps were performed repeatedly: opening documents, running Ctrl+F searches, manually checking filing dates, and taking handwritten or Word-based notes.

In our environment and using our programming language, the examiner begins by loading the candidate references into the search workspace and making the active corpus explicit. They can narrow the working set before close reading by combining metadata and classification constraints directly in the same query state that will later drive passage retrieval.

```
meta.filingDate:<2018-03-15 AND cpc:"H04L_45/00"
```

Listing 1: Scoping a legally admissible technical slice before close reading.

This first step is a direct design response to Need 2, where participants described wasting significant effort on references that turned out to postdate the application’s effective filing date. Instead of opening every candidate reference to inspect dates manually,

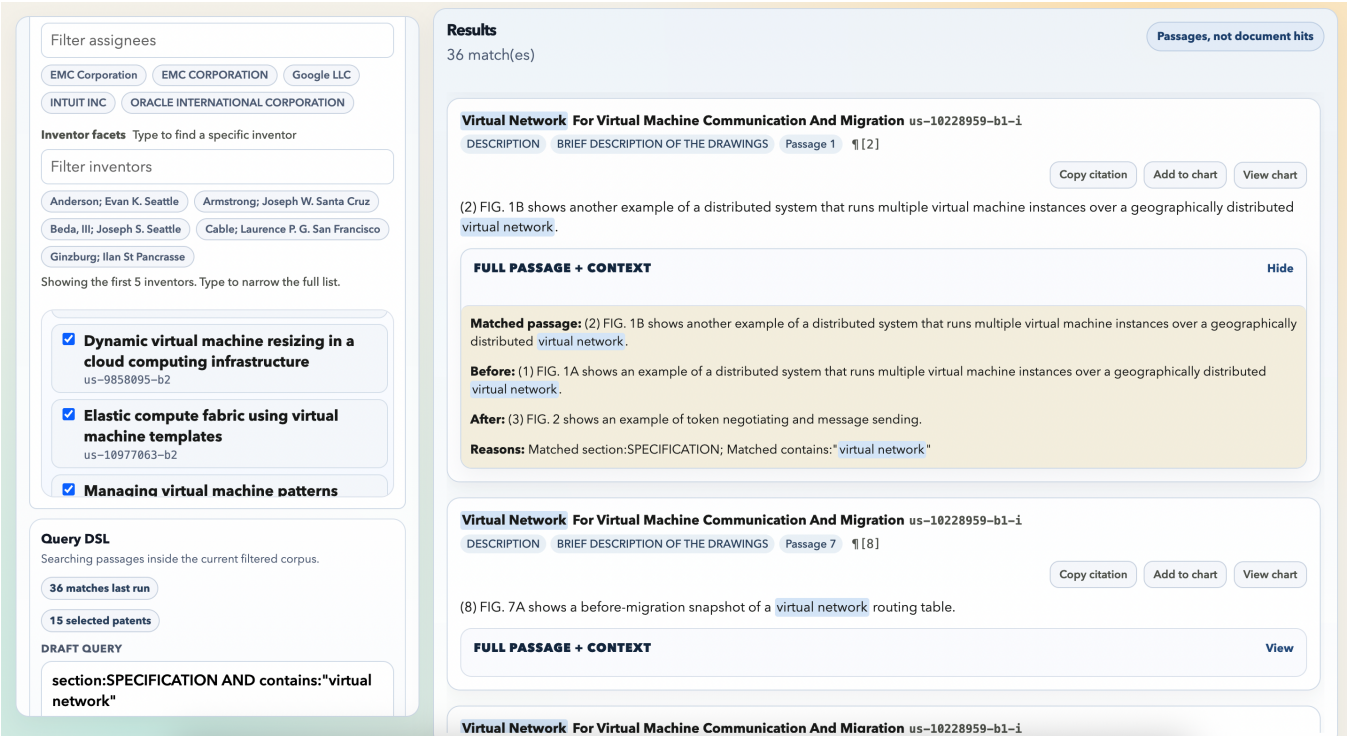


Figure 1: Search workspace for the prototype. The interface keeps corpus scoping, metadata facets, DSL query authoring, and passage-level results visible in one surface so an examiner can iteratively refine the search space, inspect local context around each hit, and preserve citation-ready evidence without reopening full patent documents in a separate viewer.

the examiner can keep the admissibility constraint attached to the active corpus and revise it as new references are added or removed. Once the corpus is scoped, the examiner searches for the claim element itself. Rather than running a separate search for each synonymous phrase, the examiner can express the synonym set and section restriction in one query over the selected documents:

```
section:SPECIFICATION AND
  (contains:"routing_table"
   OR contains:"forwarding_table"
   OR contains:"FIB"
   OR contains:"routing_cache")
```

Listing 2: Passage-level query for a claim element across a scoped corpus.

Instead of returning a ranked list of entire patent documents, the system returns a list of matching passages drawn directly from the scoped corpus. Each result corresponds to a specific passage that satisfied the query predicates, allowing the examiner to inspect candidate evidence at the level where claim mapping actually occurs rather than first navigating through full documents manually. Each card exposes the section type and heading, the paragraph or claim anchor if present, and the immediately neighboring passages. This directly addresses Need 4, in which participants reported having to open documents, run repeated keyword searches, and manually determine whether a keyword hit appeared in a meaningful section of the patent or was actually relevant in context. If a passage looks

promising, the examiner can copy a citation trace or add the result directly to the claim-chart workspace. The task then continues into synthesis rather than stopping at retrieval. If the examiner adds a result to the chart, the system preserves the passage, its anchor, the source document IDs, and the query text that produced it. The chart workspace groups evidence under claim elements and keeps rows editable, allowing the examiner to continue building a claim chart without losing the provenance of how each piece of evidence was found. This is the design response to Need 3: the handoff from search to claim mapping is treated as part of the same workflow (rather than as a manual export into Word or paper notes, where there is no unified process for examiners to follow). Figure 2 shows this downstream workspace, where saved evidence remains grouped, editable, and linked back to the originating search state. Finally, if the result set contains false positives, the examiner can refine the query incrementally instead of restarting the search process. For example, if the synonym-expanded result set still mixes dynamic rerouting with static configuration text, the examiner can add a negative constraint directly to the same query program:

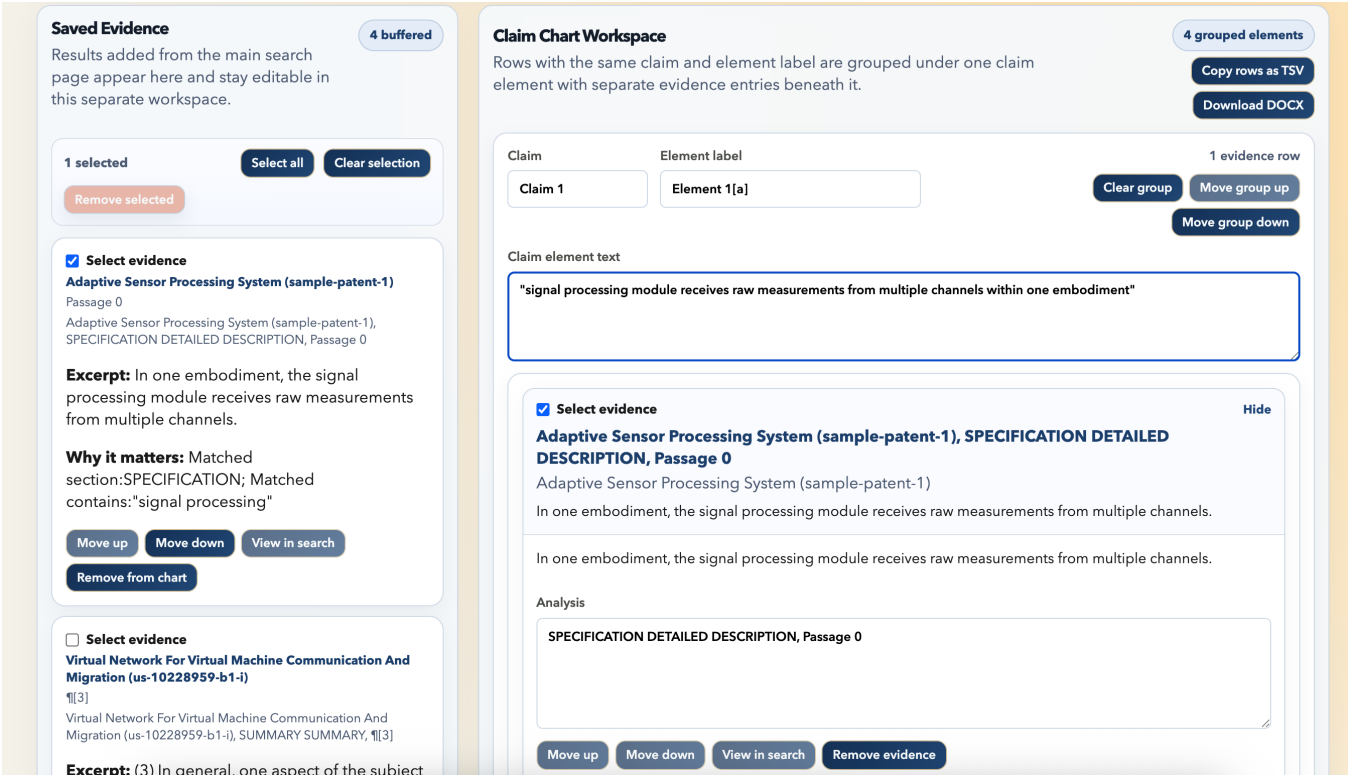


Figure 2: Claim-chart workspace for the prototype. Evidence selected during search remains available as the examiner drafts grouped claim-chart rows, edits element text, and exports the resulting chart, making the handoff from retrieval to downstream synthesis part of the same interactive workflow.

```
section:SPECIFICATION AND
  (contains:"routing_table"
   OR contains:"forwarding_table"
   OR contains:"FIB") AND
  NOT contains:"static_route"
```

Listing 3: Refining a noisy result set with explicit exclusion.

The debounced live-refresh loop immediately shows the effect of that change, so the examiner can assess whether the new constraint has removed false positives without restarting the workflow. The same DSL supports both broad passage-level retrieval across a scoped corpus and more targeted inspection within specific documents or claim sets. As examiners refine their analysis, they can combine structural anchors with content constraints to move from wide exploratory search toward pinpoint retrieval for a particular claim element or supporting passage:

```
claim:8 AND
  (contains:"routing_cache"
   OR contains:"forwarding_table")
```

Listing 4: Combining claim anchors with content constraints.

This behavior directly addresses Need 5 by supporting the iterative search strategy examiners already use in practice. Instead of forcing examiners to repeatedly restart the search process or rely on opaque similarity-search results, the DSL allows examiners to incrementally refine queries as they inspect retrieved passages and

learn more about the technical language used in the art. Examiners can narrow, expand, or restructure the same query in response to what they find, thereby keeping search an ongoing process of appraisal and refinement rather than a manual workflow in which documents must be opened and skimmed one by one to determine whether they are actually relevant. Throughout this process, the search state and retrieved evidence remain connected to the downstream claim-chart workflow rather than becoming detached notes or copied text.

4.3 Decision 1: Make corpus refinement a first-class part of the DSL

Observation from Need-Finding. Our need-finding showed that prior-art search was iterative at the corpus level: before close reading, examiners repeatedly narrowed and revisited an active working set using filing dates, classifications, and prior references. Therefore, corpus refinement was part of the task itself rather than a one-time setup step.

Design Requirement. The DSL could not treat search as flat keyword retrieval over an undifferentiated document collection. It needed to represent corpus refinement as part of the query language itself, so that examiners could explicitly construct, constrain, and revise a working set before moving into close passage-level inspection.

Implemented Response. We designed the DSL around composable metadata-aware predicates that operate over patent-level fields as well as document text. Instead of requiring examiners to manually inspect candidate references one-by-one to determine whether they fit the relevant filing-date window or technical classification, these constraints can be written directly into the query. This led to structured predicates such as `meta.filingDate:`, `cpc:`, `assignee:`, and `inventor:`, which allow corpus-scoping decisions to remain explicit, inspectable, and revisable as the search evolves.

```
meta.filingDate:<2018-03-15
AND cpc:"H04L_45/00"
AND assignee:"Google_LLC"
```

Listing 5: Constraining a working corpus using metadata-aware predicates.

This decision directly addresses Need 2 by moving filing-date validation from a separate manual check to the query language. It also supports the broader workflow pattern observed across participants: examiners repeatedly narrow, revisit, and revise a working set before close reading. By making corpus refinement part of the DSL, the system treats search scoping as an explicit design object rather than a hidden setup that happens outside the tool.

4.4 Decision 2: Preserve patent structure in the retrieval model

Observation from Need-Finding. Participants described their information needs in terms of sections and citeable locations, not whole documents. P05 wanted hits in the background and detailed description rather than the abstract or claims, while P01 emphasized the effort required to recover the exact paragraph-level location needed for citation.

Design Requirement. The retrieval unit had to be smaller than the full document and had to preserve enough structure for relevance judgment, citation, and reuse. This meant modeling patents in the same units examiners actually use when talking about evidence.

Implemented Response. We represent each patent as a Document → Section → Passage hierarchy. A Document stores metadata such as filing date, assignee, inventors, and CPC codes. A Section carries a canonical type drawn from a closed vocabulary such as BACKGROUND, SPECIFICATION, or CLAIMS. A Passage is the smallest retrieval unit and can carry anchors such as `paragraphId`, `claimNo`, and figure references. The result card then displays the section type, heading, anchors, and neighboring passages so the examiner can judge for relevance without reconstructing that structure manually.

This design choice also explains why section type is a dedicated predicate rather than an arbitrary metadata field. Participants referred to legal section roles, not heading substrings. P05 asked for hits “in the background and description sections, not in the abstract or claims,” so the model normalizes those section roles at parse time and exposes them directly in the query language:

```
section:SPECIFICATION AND contains:"virtual_machine_migration"
```

Listing 6: Section-scoped containment query.

Similarly, paragraph anchors are preserved because citation work depends on them. By extracting `paragraphId` at parse time and surfacing it on the result card, the tool keeps citation-relevant structure available during search instead of requiring the examiner to reopen the source document.

4.5 Decision 3: Make refinement inspectable through composable query expressions

Observation from Need-Finding. Participants did not formulate one perfect query and wait for a ranked list. They iteratively probed the corpus, expanded synonym sets, added or removed constraints, and revised searches as they learned more about the art. P02 described this process as “appraising the art as you search for it, and refining your search based on that.” Participants also expressed frustration with search systems whose ranked outputs did not make clear why particular results appeared or how to revise an unsuccessful search.

Design Requirement. The DSL needed to support refinement as an explicit interaction, not as an invisible ranking adjustment. Examiners needed to be able to make small local changes, inspect the effect of those changes, and understand which parts of the query caused a passage to match. This required a query representation that was editable, composable, and explainable.

Implemented Response. We implemented the DSL as a textual Boolean language whose expressions denote matching passages rather than opaque document rankings. Predicates such as `contains:`, `section:`, `cpc:`, and `meta.filingDate:` can be combined using AND, OR, NOT, and parentheses. Internally, the parser converts each query into an explicit query representation that the evaluator applies over passage candidates and records match reasons for each returned passage. As a result, the user can see not only that a passage matched, but which predicate or combination of predicates caused it to appear.

This decision is most visible in synonym refinement. Need 1 showed that examiners often manually enumerate alternate technical terms and rerun searches for each variant. Rather than treating synonym expansion as an opaque model behavior, the DSL represents it as ordinary Boolean composition:

```
section:SPECIFICATION AND
(contains:"routing_table"
OR contains:"forwarding_table"
OR contains:"FIB"
OR contains:"routing_cache")
```

Listing 7: Transparent synonym refinement through Boolean composition.

This makes the refinement state inspectable. If the query is too broad, the examiner can remove a term, add a constraint, or introduce an exclusion without abandoning the search state:

```
section: SPECIFICATION AND
  (contains: "routing_table"
   OR contains: "forwarding_table"
   OR contains: "FIB") AND
  NOT contains: "static_route"
```

Listing 8: Narrowing a query through local edits.

We explicitly considered a GUI-only visual query builder as an alternative to the textual DSL, but rejected it because it would have turned the search strategy into a transient interface state rather than a persistent, reusable artifact. Examiners already work with Boolean-style search strings, and participants described revisiting prior searches during later stages of examination. A textual query preserves the rationale behind a search in a compact form that can be edited, saved, shared, and reused as the examiner moves from broad exploration toward evidence selection. In other words, the DSL was not only an implementation convenience; it was a deliberate choice to keep search logic inspectable and reusable rather than burying it inside point-and-click controls.

4.6 Decision 4: Add `synonym_of` as a controlled synonym-expansion primitive

Observation from Need-Finding. Need 1 showed that synonym generation was a central and unsupported step in examiner search; Section 4.2 provides the full qualitative evidence. For the design problem here, the key issue was that examiners needed help expanding technical terms without turning that process into opaque automation.

Design Requirement. The DSL needed to support synonym expansion as a first-class search operation, not merely as informal advice or a hidden semantic-search feature. Because participants relied on synonym discovery to translate claim language into searchable prior-art terminology, the system needed a way to help examiners broaden a term while still keeping the expansion inspectable, editable, and compatible with ordinary Boolean refinement. The central design tension was between automation and transparency: a fully manual approach would preserve control but recreate the burden of assembling long synonym lists by hand, while a fully opaque expansion model would hide which terms shaped the result set and make it difficult to diagnose noise. The operator therefore needed to automate suggestion while still exposing enough control for examiners to steer or bound the expansion when it drifted away from the intended technical sense.

Implemented Response. We added `synonym_of`: as a controlled synonym-expansion primitive in the DSL. Instead of forcing examiners to manually reconstruct long OR chains whenever they want to search for alternate technical phrasings, the examiner can write a seed term directly in the query language:

```
synonym_of: "routing_table"
```

Listing 9: Controlled synonym expansion through a dedicated predicate.

To keep this expansion examiner-controllable, we also included optional arguments that bound and steer the synonym behavior. The `max` option limits the number of generated or used synonyms, while `topics` lets the examiner bias the expansion toward a technical domain. These parameters are not just convenience

settings; they are the mechanism by which the design resolves the transparency-versus-automation tension. `max` prevents the operator from silently broadening a query into a large, noisy bundle of loosely related terms, and `topics` gives the examiner a way to indicate which technical sense of an ambiguous term should dominate the expansion. For example:

```
synonym_of: "node" | max=10 | topics="computer_science_and_system_
networking"
```

Listing 10: Constraining synonym expansion with count and topic options.

These options were important because Need 1 was not simply a request for automatic synonym generation. Participants described synonym work as context-sensitive: terms such as “node” can mean different things depending on the technical field, and adding too many loosely related alternatives can make the search noisier rather than better. In other words, the goal was not to guess synonyms for the examiner and hide the guesswork, but to let the system participate in expansion while keeping the resulting search behavior legible. By exposing `max` and `topics` in the DSL, the examiner can control both the breadth and direction of expansion instead of accepting an unbounded synonym list. This keeps synonym assistance programmable and revisable inside the same query artifact as the rest of the search strategy.

Internally, `synonym_of: "routing_table"` is rewritten into ordinary `contains`: predicates, preserving compatibility with the existing Boolean evaluation model:

```
(contains: "routing_table"
 OR contains: "routing_protocol"
 OR contains: "interior_gateway_protocol"
 OR contains: "routing_cache")
```

Listing 11: Possible internal expansion of `synonym_of`.

This decision directly responds to Need 1 because it turns synonym discovery from an ad hoc mental or note-taking activity into an explicit query-language operation. The important design choice was not simply to add more terms automatically; it was to make synonym expansion controllable within the same DSL that examiners use for iterative search. If an expansion introduces noisy terms, the examiner can inspect and revise the resulting alternatives. If a retrieved passage reveals a better phrase, that phrase can be added to the query and reused in later searches. In this way, `synonym_of`: preserves the examiner’s existing strategy of concept translation and iterative term discovery while reducing the burden of repeatedly generating synonym sets by hand.

4.7 Decision 5: Keep retrieval and claim mapping in one provenance-preserving workflow

Observation from Need-finding. Need 3 showed that the transition from search to claim mapping was largely manual and ad hoc. Retrieval did not end when a participant found a promising passage; participants still had to move evidence into Word documents, personal claim charts, or paper notes while preserving enough citation detail to support later rejection writing. P03 described first finding relevant prior art and then mapping it back to claim language, explaining that they would “write first, then map to prior art.” This

made evidence capture a separate reconstruction step rather than a continuation of search. Similarly, P01 emphasized the burden of recovering the exact paragraph-level location needed for citation after identifying useful material, showing that the problem was not only finding relevant text but keeping that text connected to its source, anchor, and search context.

Design Requirement. The tool had to preserve provenance across the handoff from retrieval to synthesis and had to treat evidence capture as part of the same workflow rather than an external afterthought.

Implemented Response. Each result card includes Copy citation and Add to chart actions. When a passage is transferred, the system keeps the selected passage, its anchor, the source document set, and the query that produced it. The claim-chart workspace then groups evidence under claim elements, keeps rows and underlying analysis editable, and supports export as TSV or DOCX. DOCX export was included specifically because nearly every participant in our study performed claim mapping in Word documents, making it the most natural handoff format for the downstream office-action workflow.

This decision extends the tool from a custom DSL-backed retrieval prototype into a workflow-oriented prototype. In the P03-style task we observed, the examiner did not only need to find passages mentioning a routing-table concept. They also needed to judge those passages in context, preserve paragraph-level evidence, and organize the results under a claim element without losing where the evidence came from. The search workspace therefore presents neighboring context and anchors on the result card, while the chart workspace carries selected results forward as evidence artifacts rather than detached text snippets. In this sense, the charting feature demonstrates why provenance matters beyond retrieval: it preserves enough state to support the first handoff from search to claim mapping without forcing examiners to manually reconstruct which corpus was searched, which query produced the passage, where the passage appeared, or how it should be cited.

5 Technical Challenges and Technical Solutions

The hardest implementation problems in this project came from turning a messy, document-centered examiner workflow into a programmable system with stable objects, meaningful queries, and reusable results. Patent search is not only a keyword retrieval problem: examiners need to search across semi-structured legal and technical documents, interpret results in context, preserve citable anchors, and carry useful passages into downstream claim analysis without losing where those passages came from.

Primarily two connected technical challenges shaped the implementation. The first was recovering a structured document model from heterogeneous patent sources so that patent-specific DSL queries would have a stable database to search on. The second was preserving search results as reusable evidence artifacts rather than allowing them to disappear as ordinary search hits once the user moved into claim mapping. Together, these challenges define the core abstraction of the prototype. Patents are represented as structured document objects, queries operate as executable expressions over those objects, and results preserve the provenance needed to support later evidence use.

5.1 Challenge 1: Recovering Structured, Queryable Passages from Noisy Patent Sources

The first challenge was about both parsing and representation. As we parsed more patents, we found that patent sources do not all arrive as clean databases as some documents are available as plain-text files with explicit section markers and paragraph anchors, while others are only available as text-extractable PDFs whose formatting is noisier and whose logical structure is only partially visible in the extracted text stream. A passage-level query language would only be meaningful if these varied sources can be normalized into the same internal representation. Otherwise, a query such as `section:SPECIFICATION, paragraph:42, or claim:1` would mean different things depending on the original document format.

The technical problem was therefore not just extracting text, but defining a common semantic domain for the rest of the system. Examiner-style queries often combine constraints that naturally live at different levels of a patent document. A phrase filter such as `contains:"virtual machine"` applies to passage text; a section filter such as `section:CLAIMS` applies to the passage's enclosing section; and a metadata filter such as `meta.pubDate:>=2019-01-01` applies to the enclosing document. Users, however, do not experience these as separate query systems. They expect to write one query that combines all of them and receive passage-level results that remain tied to the relevant section and document context.

A naive implementation could have kept separate parsing paths for text files and PDFs, allowing each parser to emit whatever structure it could recover. That would have been easier to prototype, but it would have pushed source-specific uncertainty into every later stage of the system. Structural predicates, metadata filters, evidence anchors, and claim-chart references would all become dependent on the quirks of the original file format.

Our solution was to normalize all sources into a shared Document $\rightarrow$ Section $\rightarrow$ Passage hierarchy before query execution. The parser canonicalizes heading variants into a closed section vocabulary, splits document text into passage units, and annotates passages with available structural anchors such as `paragraphId`, `claimNo`, and figure references. Each passage also retains links to its enclosing section and source document. This makes the hierarchy the semantic domain of the DSL, where the same structural query should refer to the same kind of object whether the source was a USPTO-style text file or a PDF extracted through pypdf.

More formally, the parser must maintain a representation invariant that makes the DSL format-independent. For any source $s \in \text{Sources}$, parsing must produce a hierarchy in which every retrieved passage has a unique enclosing section and document, and in which structural predicates denote the same kind of object regardless of source format:

$$\forall s \in \text{Sources}, \text{parse}(s) = \langle D, S, P \rangle \quad \text{such that} \quad \forall p \in P, \exists! s_p \in S, \exists! d_p \in D$$

with $p \in s_p$ and $s_p \in d_p$, and with predicate interpretation defined uniformly over that hierarchy:

$$\begin{aligned} \llbracket \text{section}:t \rrbracket(p) &\iff \text{sectionType}(s_p) = t, \\ \llbracket \text{paragraph}:n \rrbracket(p) &\iff \text{paragraphId}(p) = n. \end{aligned}$$

In other words, a query such as `section:SPECIFICATION AND paragraph:42` should denote the same kind of object whether the underlying source came from a raw text file or a text-extracted PDF.

This normalized representation also gives mixed-granularity queries a concrete execution strategy. Query execution can flatten the Document $\rightarrow$ Section $\rightarrow$ Passage hierarchy into candidate passages while still preserving each passage's section and document links. Passage-level predicates are evaluated against the passage text, section-level predicates against the enclosing section, and document-level predicates against the enclosing document. For example, `meta.pubDate:>=2019-01-01 AND section:CLAIMS AND contains:"virtual machine"` is evaluated as a passage query, but each candidate passage carries enough context for all three constraints to be interpreted correctly.

This design choice simplified the rest of the architecture. Because the normalized hierarchy is stored as structured JSON rather than reconstructed on every query, the query engine and evidence layer can operate over one document model even when the original sources differ substantially. The tradeoff is that parsing becomes a semantically important stage of the pipeline, and if the parser misidentifies a section boundary or fails to recover a paragraph anchor, the downstream query result inherits that limitation. However, making that structure explicit is still preferable to treating patents as flat text, because it gives the system a stable object model that can be inspected, improved, and reused.

5.2 Challenge 2: Preserving Search Results as Reusable Evidence

The second challenge was making query results reusable as evidence rather than treating them as standalone search hits. A matched passage is only useful to an examiner if it can be inspected in context, tied back to a citable location, and later reused in claim analysis. USPTO's search tool separates these tasks: search returns a hit, while note-taking, citation, and claim mapping happen somewhere else. Our need-finding study suggested that this split is itself a source of workflow friction because the user must manually reconstruct why a passage mattered after leaving the search interface.

The technical problem was therefore to define a result object rich enough to support immediate relevance judgment, later citation, and downstream reuse. A bare text snippet is insufficient because it omits the structural anchor, surrounding context, and query state that produced the match. It also omits the explanation for why the passage was matched in the first place. This matters because patent search is iterative: users often revise a query after seeing whether a passage matched for the intended reason. If the system only preserves the passage text, then the user loses part of the reasoning context needed to evaluate and reuse that result.

Our solution was to preserve provenance end to end. Each query match carries its document ID, section type, section title, passage index, optional paragraph or claim anchor, neighboring context, and human-readable match reasons derived from the predicates that matched. Since our DSL's queries are evaluated through explicit predicates rather than vague similarity scores, the system can report lightweight explanations such as whether a result matched because

it appeared in a target section or came from a document satisfying a metadata constraint.

When a user adds a result to the claim-chart workspace, the stored row records information basic information such as the source query text. At the same time, backwards traceable information such as source document IDs, source result key, and passage citation is preserved. Therefore a row in the chart is not only an excerpt, but a reference back to the executable query state that produced the excerpt. That provenance-preserving representation is what makes the search and synthesis workflow inspectable and reversible.

More precisely, a result is modeled as a structured value:

$$\text{Match} = \left\langle \begin{array}{l} \text{passageId, documentId,} \\ \text{sectionType, sectionTitle,} \\ \text{passageIndex, passageText,} \\ \text{contextBefore, contextAfter,} \\ \text{reasons, paragraphId?,} \\ \text{claimNo?, figureRefs?} \end{array} \right\rangle$$

The evidence object carried into downstream synthesis is not the full match object. Instead, it is a provenance-preserving projection of the match, extended with enough query state to navigate back to the original result:

$$\text{Evidence} = \text{ChartFields} \times \text{QueryText} \times \mathcal{P}(\text{DocumentId}) \times \text{ResultKey}$$

where *ChartFields* are the fields needed by the claim-chart workflow: a stable row id, citation, reference, location, excerpt, match reason, editable claim and element fields, and notes. In the implementation, this corresponds to the chart row:

$$\text{ChartRow} = \left\langle \begin{array}{l} \text{id, claim, elementLabel,} \\ \text{reference, location, citation,} \\ \text{excerpt, reason,} \\ \text{elementText, notes,} \\ \text{sourceDocumentIds, sourceQueryText,} \\ \text{sourceResultKey} \end{array} \right\rangle$$

This representation does not preserve every field from the original backend match. For example, the chart row does not store *contextBefore*, *contextAfter*, or *figureRefs* as separate structured fields. However, it does preserve enough information to cite the evidence, display it in the chart, and return to the search state that produced it.

Evidence is more than just "text plus notes," it is a query result paired with enough provenance to be rechecked, regrouped, or exported without severing it from the search process that produced it. This is also where the prototype becomes more than a retrieval tool. Once result objects preserve enough state to be cited, revisited, and reorganized under claim elements, they can become domain artifacts in their own right.

However, the current representation is still limited. Explanations report matched predicates, but they do not yet provide a full proof tree for complex nested queries. The evaluator is also Boolean rather than ranked, so it can determine whether a passage matches but not whether it is the best passage among many possible matches. Still, preserving reasons, anchors, context, and source query state gives

the prototype a stronger evidence model than ordinary keyword search.

5.3 Future Work

The prototype remains incomplete in several technically important ways. First, synonym support remains underdeveloped. The current system supports explicit Boolean enumeration and uses the Datamuse API [23] to retrieve related terms for a user-provided query term, but it does not yet provide a robust domain-specific mechanism for discovering terminology variants. A stronger implementation would recursively traverse the selected patent documents to identify candidate synonyms, related technical phrases, and alternate descriptions used across specifications, claims, and abstracts. This would better match the workflow observed in Need 1, where examiners described developing search vocabulary by inspecting specifications, prior references and knowledge, and related terminology rather than relying on a fixed synonym list.

Second, the current DSL implements only a limited set of patent-specific filters and operators. With more time, we would have liked to add more examiner-oriented constructs, such as richer admissibility filters, more precise claim-reference filters, and stronger operators for proximity variation. While the current DSL already captures the core structure of examiner workflows, these additions would strengthen that foundation by covering more specialized search practices. At the same time, they would need to be designed carefully so that the language remains transparent rather than becoming a large collection of disconnected special cases.

Third, the claim-chart workspace preserves provenance, but claim-element grouping and evidence reuse are still encoded mainly in UI state rather than in a more explicit programmable representation. A future version should make claim elements, evidence groups, and rejection rationales first-class data objects. That would allow the system to support more of the downstream synthesis workflow while preserving the same provenance guarantees used in search.

Finally, the prototype is only designed for an MVP-sized corpus, not a production-level corpus or a live USPTO-scale workflow. Currently, it separates lightweight corpus discovery from full passage-level query execution through preprocessing of patent documents into structured JSON, while startup indexing uses smaller metadata records so the interface can list and filter documents without loading every passage body. Full documents are loaded only when selected for querying or preloading, then cached for later use. This improves responsiveness for the current corpus, but the system is not yet a full large-scale retrieval architecture. We also planned, but did not implement, integration with actual USPTO APIs or examiner-facing tooling, so the current system operates over a local, hand-downloaded and parsed corpus rather than live patent database sources. Scaling the prototype would therefore require both more aggressive indexing and caching, and a real ingestion/integration layer for USPTO data sources or tools.

6 Implementation

The prototype is implemented as a Python 3/FastAPI backend and a React 18/TypeScript/Vite frontend. Raw patent sources are stored in `backend/data/raw/` and parsed into structured JSON documents in `backend/data/parsed/`; text-extractable PDFs are read with

`pypdf`, while custom parsing code extracts patent metadata, sections, passages, claim numbers, paragraph anchors, and figure references into a normalized Document → Section → Passage representation. The backend exposes API endpoints for document listing, document preloading, raw-file parsing, synonym-set inspection, and DSL query execution. Internally, query strings are parsed by a custom recursive-descent parser into typed Pydantic expression objects, including Boolean expression nodes and leaf filters for section, containment, metadata, CPC, paragraph, claim, and figure constraints. Query execution flattens selected documents into passage candidates and evaluates the expression tree over those candidates, returning structured matches with context, anchors, and match reasons. Synonym support is implemented by expanding `synonym_of` clauses through the Datamuse API [23] into ordinary contains filters and saving reusable in-memory termset expansions. For MVP-scale performance, the backend stores lightweight metadata sidecar files, indexes those records on startup, lazy-loads full parsed documents only when selected for querying or preloading, and caches hydrated documents in memory.

The frontend acts as a thin client over the backend APIs, providing document selection, query editing, live refinement, result inspection, and claim-chart management; the chart export path uses the `docx` package to generate lightweight office-action-style artifacts. We implemented pytest-based tests for the parser, DSL, query engine, source loader, synonym logic, document store, and API surface. The full implementation and setup instructions are available at <https://github.com/shanayamalik/cs294-sp26>.

7 Discussion

The main lesson from this project is that the PL contribution is not merely “adding a DSL” to an existing interface, but choosing an abstraction boundary that matches the full domain workflow. By re-representing patent documents as structured objects and search results as durable evidence objects, we created operations that flat document-level search and snippet-only result views cannot express cleanly. In our implementation, this became clear whenever a feature depended on passage-level behavior or on carrying a result forward into claim mapping. Section filters, local context, citation anchors, and provenance-preserving chart rows all worked because the underlying representation treated them as meaningful parts of the task rather than as display details layered on later. A concrete mistake a tool builder could make is to treat the first workable surface model as the right semantic model: documents as opaque text, results as visible snippets, and downstream organization as UI state to be added later. In our project, every new feature made the cost of that shortcut more visible. Features like section filters, neighboring context, stable anchors, and reusable evidence rows all became harder to support when the system did not already model the objects examiners reason about. Our experience suggests that representation choices made early in the build largely determine which language features feel natural and which become awkward special cases. A more reliable design move is to test the representation against an end-to-end task before polishing the interface: if the core model cannot express the domain queries users need or carry results forward without fallback logic and manual reconstruction, the abstraction boundary is probably wrong.

The second lesson is methodological. Our contextual inquiry did not just help us choose features, it helped us translate examiner needs into concrete design and implementation decisions. A concrete mistake a future implementer could make is to treat need-finding as a checklist of features rather than as evidence about where the current workflow breaks down. In our case, the important observations were not simply that examiners wanted better search. They were repeatedly doing extra work to manage synonym choices, filing-date relevance, passage context, and claim-mapping evidence across disconnected tools. Those recurring points of friction helped us decide what the prototype needed to support first. Features such as section-aware search and passage-level results were not arbitrary technical features but rather they were implementation responses to specific needs we saw in the study. This became especially important while building the tool because some ideas that sounded useful in the abstract only became worth implementing when we could connect them to a concrete workflow problem. For example, passage-level retrieval mattered because examiners needed usable evidence, not just relevant documents. Similarly, saved evidence rows mattered because the search process did not end when a result appeared; the result still had to be interpreted, preserved, and organized under a claim element. The lesson for future language and environment builders is to trace a real task from beginning to end and identify where users repeatedly face friction with information or data. We've found these moments often provide stronger design guidance than general feature wish lists as they lead to features that concretely reduce a specific burden observed in users' workflows.

The third lesson concerns transparency. When tools are built for expert users, especially in legal or legal-adjacent work where small wording and citation differences can matter, the system should make its actions easy to inspect rather than asking users to trust an unexplained output. A concrete mistake a tool builder could make is to treat transparency as a surface-level feature, such as adding highlights or short explanation text after the result has already been produced. In our project, we learned that transparency had to be connected to the actual search process. Even simple match-reason information was useful only when it accurately reflected why a result appeared, such as the section searched, the passage matched, or the query condition that applied. This mattered because examiners would not only be using results to find relevant documents, but they would also need to judge whether a passage could be carried forward as evidence. During implementation, this made us more careful about preserving source information, anchors, query context, and result explanations together rather than treating them as separate details. At the same time, making this explicit is not free. It shifts complexity from the user's manual process into the language design, data model, and interface, so future implementers need to decide carefully which parts of the workflow deserve to become first-class objects. In our project, sections, passages, anchors, query context, and evidence rows earned that treatment because they repeatedly appeared in the examiner's actual workflows. The lesson here is that expert-facing tools should make their reasoning visible in ways that support careful review, correction, and reuse, while still remaining simple enough for users to understand and revise.

Taken together, these lessons point to a broader design stance: language-oriented tools for expert work are strongest when they make the key workflow objects explicit, durable, and inspectable.

8 Additional Notes

8.1 What the P03 Task Revealed About Workflow State

In order to ground the design in concrete practice, it helps to trace what changed when we replayed a P03-style task through the prototype. The earlier sections describe the system in terms of its components — corpus scoping, passage-level querying, provenance-preserving results, and claim-chart capture. Replaying the P03 task reframes those same components from a workflow perspective: the central problem was not that P03 needed faster search, but that critical workflow state kept dissolving as work moved across tools, leaving examiners to reconstruct context by hand at each transition.

In the observed workflow, P03 moved among classification-based searching, keyword refinement, filing-date checks, foreign-art inspection, and claim mapping. Each step produced information that mattered later, but much of that information lived only in working memory, temporary search history, or manually copied notes. The task was therefore not a single search followed by a single judgment; it was a sequence of revisions in which the active corpus, admissibility constraints, search vocabulary, and candidate evidence kept changing together. This helped us understand why the prototype needed to make corpus state explicit: selected documents, metadata constraints, and query text had to remain tied together so the examiner could inspect and revise the working set rather than reconstruct it from memory.

The P03 task also clarified why passage-level search mattered. P03 did not just need to know whether a document contained relevant keywords; they needed to decide whether the words appeared in a meaningful context and could support a claim-mapping decision. In the prototype, result cards do not eliminate that close reading, but they move the first meaningful relevance judgment closer to the moment of retrieval by showing section, anchor, and neighboring context together. The system is therefore not trying to decide relevance for the examiner, but to reduce the amount of reconstruction required before the examiner can judge relevance.

Finally, the P03 replay showed why the handoff from search to claim mapping should be treated as part of the same workflow. Once useful art is found, the examiner still needs to preserve evidence, connect it to claim language, and remember where it came from. That observation shaped our decision to carry selected results into the claim-chart workspace as evidence objects rather than detached snippets. Seen this way, the prototype is best understood as an attempt to stabilize the most fragile state transitions in the examiner workflow. It makes the active corpus more explicit, brings passage context closer to retrieval, and keeps selected evidence connected to the search state that produced it, while also making clearer what remains incomplete.

8.2 Why We Did Not Center the Tool on Semantic Search or LLMs

Semantic search and large language models (LLMs) might seem like natural tools for patent search: examiners struggle with synonyms, claim language often differs from prior-art terminology, and relevant references may use unexpected phrasing. Our need-finding suggested a more nuanced picture; Need 5 in particular showed that examiners were not simply struggling to find relevant documents — they were struggling to control, inspect, and preserve the reasoning state behind a search within time-limited workflows. A standalone semantic search result or LLM response does not naturally provide that kind of stable, editable representation. The problem was not that existing tools lacked intelligence; it was that they lacked structure and traceability in how results were surfaced and carried forward.

P03's comments on AI-assisted search reinforced this point. The USPTO already provides an internal similarity-search tool, but P03 described it as only occasionally useful and not reliable enough to replace manual refinement. P03 also described cases where searches surfaced topically mismatched results, such as vehicle-related queries returning unrelated domains because the terms overlapped without preserving the intended context. This matters because the problem our project addresses is not simply the absence of semantic search. It is the lack of an integrated and controllable workflow that connects corpus scoping, query refinement, passage inspection, and evidence capture. If an AI system returns a plausible reference without making the search state, source context, and downstream evidence relationship clear, the examiner still has to reconstruct much of the reasoning by hand.

External LLMs also introduce practical constraints that were outside the scope of this project. Patent examination often involves unpublished or restricted application materials, so privacy is not optional. P03 noted that tools such as ChatGPT could only be used when the relevant application material was already public; otherwise, the material could not simply be moved into a general-purpose AI system. A privacy-preserving LLM design would likely require local inference or a controlled internal deployment, along with additional work around model selection, compute requirements, latency, evaluation, and output traceability. Those were not resources or topics we were looking to engage in for this prototype, and they were not the main research question we wanted to answer.

This is why the prototype centers on a DSL-backed workflow. The research argument for the DSL is that it provides an inspectable, editable representation of search intent that examiners can revise and defend as they move from corpus scoping to passage inspection and evidence capture. Compared with an LLM-centered system, a DSL was also more realistic for our project scope because it is far less computationally demanding and far faster to run. More importantly, it let us investigate whether a structured language could better support the search-to-evidence process than a system whose reasoning remained primarily implicit.

For these reasons, we treated AI as a possible support mechanism rather than the foundation of the prototype. Future versions could use LLMs for small, supportive tasks such as suggesting synonyms, summarizing neighboring context, or recommending broader search directions. However, those suggestions would be

most useful if they became visible, editable parts of the query and evidence state rather than black-box generated responses. Our goal was not to reject semantic search or LLMs, but to avoid making them the only place where reasoning happens. In this framing, AI could eventually help produce better candidate terms or passages, but the system still needs an inspectable representation that examiners can revise and defend.

8.3 Limitations and Threats to Validity

Several limitations are worth noting for readers who wish to build on this work. First, the system has been informed by contextual inquiry and implemented as a working prototype, but a follow-up evaluation with patent examiners using the full tool in practice has not yet been conducted. The paper therefore supports claims about observed workflow needs, design rationale, and technical properties of the implementation, while leaving questions about productivity gains, preference, and long-term adoption open for future work. Second, the participant pool is small and focused on technical patent examination, which is appropriate for a need-finding study but means the findings are best read as design-relevant observations for this domain rather than as a comprehensive account of all patent examination practice. Third, as discussed in Section 5.3, the prototype currently operates on a small parsed corpus. The design decisions around metadata indexing, document preloading, and passage-level query execution have worked well at MVP scale, but have not yet been tested at the scale of production patent search systems, which may involve collections orders of magnitude larger.

There is also a methodological limitation, which is that our need-finding data come from observed workflows around public patents and reconstructed search activity, not from direct instrumentation of proprietary examiner systems under live production conditions. This approach was appropriate given the access and ethical constraints of the study, but it means there are likely additional organizational constraints, collaboration practices, and policy-driven behaviors that the prototype does not yet model. For the research community, the main implication is that this work should be read as a grounded design intervention, not as the final word on patent search tooling.

9 Conclusion

This paper presented a domain-specific query language and integrated environment for structured prior-art search, motivated by the finding that patent examiners need support not only for retrieving documents but for preserving structured evidence across the broader workflow. Through contextual inquiry with current and former patent examiners at the USPTO, we identified recurring breakdowns around synonym management, filing-date filtering, section-aware passage inspection, iterative query refinement, and the handoff from search to claim mapping. That need-finding contribution matters in its own right because it characterizes workflow constraints that retrieval-centered tools leave unsupported and provides a basis for future tool building in this domain. These findings led us to design a domain-specific query language and integrated environment that treats patents as structured and searchable objects rather than opaque full-text documents.

The prototype demonstrates how this representation can support a different kind of patent-search workflow. Examiners can scope a working corpus, write structured passage-level queries, inspect results with section and citation context, and carry selected passages into a lightweight claim-chart workspace without severing them from their source query or document provenance. In this sense, the project's main contribution is not only a DSL containing a set of patent-specific filters, but a language-oriented design approach for making expert document work more explicit, inspectable, and reusable.

The broader lesson is that tools for expert search should preserve the context that makes retrieved information useful beyond the moment of retrieval. For patent examination, this means keeping documents, queries, passages, anchors, and evidence artifacts connected as part of the same workflow. While the current prototype remains limited in scale and language coverage, it shows how a DSL-backed environment can better align with the way examiners already search, interpret, and organize prior-art evidence.

Acknowledgments

We thank Professor Sarah E. Chasins for her guidance throughout this project and the CS 294 course at UC Berkeley. We also thank the ten patent examiners who generously shared their time, expertise, and workflows with us.

References

- [1] M. Lupu, K. Mayer, N. Kando, and A. J. Trippe. *Current Challenges in Patent Information Retrieval*, volume 29 of *The Information Retrieval Series*. Springer, 2013.
- [2] P. Mahdabi and F. Crestani. The Effect of Citation Analysis on Query Expansion for Patent Retrieval. *Information Retrieval*, 17(5–6):412–429, 2013.
- [3] P. Pirolli and S. Card. The Sensemaking Process and Leverage Points for Analyst Technology as Identified Through Cognitive Task Analysis. In *Proc. Int. Conf. Intelligence Analysis*, volume 5. MITRE, McLean, VA, 2005.
- [4] D. M. Russell, M. J. Stefik, P. Pirolli, and S. K. Card. The Cost Structure of Sensemaking. In *Proc. CHI '93*, pages 269–276. ACM, 1993.
- [5] G. Marchionini. Exploratory Search: From Finding to Understanding. *Communications of the ACM*, 49(4):41–46, 2006.
- [6] M. J. Bates. The Design of Browsing and Berrypicking Techniques for the Online Search Interface. *Online Review*, 13(5):407–424, 1989.
- [7] C. C. Kuhlthau. Inside the Search Process: Information Seeking from the User's Perspective. *Journal of the American Society for Information Science*, 42(5):361–371, 1991.
- [8] R. W. White and R. A. Roth. *Exploratory Search: Beyond the Query-Response Paradigm*. Morgan & Claypool, 2009.
- [9] M. A. Hearst. *Search User Interfaces*. Cambridge University Press, 2009.
- [10] A. J. Ko, B. A. Myers, M. J. Coblenz, and H. H. Aung. An Exploratory Study of How Developers Seek, Relate, and Collect Relevant Information during Software Maintenance Tasks. *IEEE Trans. Softw. Eng.*, 32(12):971–987, 2006.
- [11] M.-A. D. Storey, F. D. Fracchia, and H. A. Müller. Cognitive Design Elements to Support the Construction of a Mental Model during Software Exploration. *J. Systems and Software*, 44(3):171–185, 1999.
- [12] N. J. Nersessian. *Creating Scientific Concepts*. MIT Press, 2008.
- [13] F. Piroi and V. Zenz. Evaluating Information Retrieval in the Intellectual Property Domain: The CLEF-IP Campaign. In *Multilingual Information Access Evaluation I. Text Retrieval Experiments*, pages 87–108. Springer, 2011.
- [14] W. Magdy, J. Leveling, and G. J. F. Jones. Exploring Structured Documents and Query Formulation Techniques for Patent Retrieval. In *Multilingual Information Access Evaluation I. Text Retrieval Experiments*, pages 410–417. Springer, 2010.
- [15] D. D. Chamberlin and R. F. Boyce. SEQUEL: A Structured English Query Language. In *Proc. ACM SIGFIDET Workshop on Data Description, Access and Control*, pages 249–264. ACM, 1974.
- [16] S. Boag, D. Chamberlin, M. F. Fernández, D. Florescu, J. Robie, and J. Siméon. XQuery 1.0: An XML Query Language (Second Edition). W3C Recommendation, 2010.
- [17] S. Evert and A. Hardie. Twenty-first Century Corpus Workbench: Updating a Query Architecture for the New Millennium. In *Proc. Corpus Linguistics 2011*, University of Birmingham, UK, 2011.
- [18] B. Piwowarski and G. Dupret. Evaluation in (XML) Information Retrieval: Expected Precision-Recall with User Modelling. In *Proc. SIGIR '06*, pages 260–267. ACM, 2006.
- [19] H. Beyer and K. Holtzblatt. *Contextual Design: Defining Customer-Centered Systems*. Morgan Kaufmann, 1997.
- [20] J. Lubin and S. E. Chasins. How Statically-Typed Functional Programmers Write Code. *Proc. ACM Program. Lang.*, 5(OOPSLA):Article 155, 2021.
- [21] T. Zhang, B. Hartmann, M. Kim, and E. Paez. Enabling Data-Driven API Design with Community Usage Data: A Need-Finding Study. In *Proc. CHI '20*, pages 1–13. ACM, 2020.
- [22] V. Braun and V. Clarke. Using Thematic Analysis in Psychology. *Qualitative Research in Psychology*, 3(2):77–101, 2006.
- [23] Datamuse. Datamuse API. <https://www.datamuse.com/api/>. Accessed May 15, 2026.